

2.6 Cryptography

Cryptography is the technique of securing information from unauthorized users by using mathematical algorithms. It enables secure communication by ensuring that only authorized parties can access or read the information and ensure that no unauthorized parties can manipulate the information. Cryptography achieves the following:

Confidentiality: The information shared through an open channel by the authorized parties cannot be read by unauthorized parties.

Integrity: The information that is stored or shared cannot be modified by any unauthorized parties without detection.

Non-Repudiation: A sender of a message or information cannot deny that they sent it.

Authentication: The origin of the message sender can be verified. For example, Ram sends the message to Shyam and Shyam can verify that the message is coming from Ram not from someone who claims to be Ram.

Blockchain uses cryptography to achieve several security properties including immutability, ownership verification, privacy, etc. The immutability is ensured using the cryptographic hash algorithms, ownership proving and verification is through the public key cryptography and digital signature. Cryptography is also used in the block growth rate control. The Proof of Work (PoW) based implementations use cryptographic hashing algorithms to control the rate. Cryptography also contributes to privacy and confidentiality in some blockchain systems, although public blockchains typically provide pseudonymity rather than complete privacy. Also, Layer 2 Smart Contracts heavily use cryptography algorithms to achieve privacy-preserving verification of proofs.

2.7 Proof of Work

Proof of Work (PoW) is the protocol which is used in blockchain implementations such as Bitcoin and the earlier version of Ethereum to control the growth of blocks in the chain as well as to achieve consensus. In PoW based consensus, nodes accept the chain with the most cumulative difficulty and discard the other branch. The first miner to solve the puzzle, called the leader, earns the right to propose the next block. It is possible that more than one leader can exist for the same block number; however, cumulative difficulty will be used to find the acceptable block and its producer as the leader. Proof of Work (PoW) uses a cryptographic hash algorithm-based puzzle, which is hard to solve, and no shortcut method exists to solve it in less time. The term Proof of Work describes that the prover proves to the verifiers that he/she/it has done the required work to solve the puzzle.

The Bitcoin Proof of Work (PoW) mechanism is based on the Hashcash system proposed by Adam Back in 1997. Hashcash was originally designed to reduce email spam and help to mitigate Denial-of-Service (DoS) attacks by requiring the sender to perform a small amount of computational work before sending an email. This computational cost discourages attackers from generating a large number of requests or spam emails. Bitcoin adopts this concept in PoW by forcing miners to solve a cryptographic hash-based puzzle before

creating a new block. The Proof of Work mechanism is used to regulate the rate of block generation and enable decentralized consensus among participating nodes.

In Bitcoin Proof of Work, a miner has to perform a computationally heavy task to convince the members of the network that he/she spent time and resources and mined the block successfully. The block will be considered part of the main chain only when it is accepted by a majority of the users or nodes in the network. The main chain is the one with more cumulative difficulty when compared with the fork, as shown in Figure 4.

Figure 4: Main Chain Selection using Cumulative Difficulty

The major advantage of the Proof of Work is that the miner can mine the block and broadcast it in the network for approval without waiting for anyone's consent to create the block. Whether the mined block will be accepted by the majority or not is the next consideration, but every miner is free from others to mine and broadcast the block. The block will be successful if it is accepted by the majority of the network members; otherwise, it will go into an orphaned fork. The process of a miner mining a block using Proof of Work is as follows:

A miner chooses the transactions that are available in their transaction pool to include in the block. The transaction pool contains many transactions, but the miner can choose only a limited number due to the restriction of the block size. In such a scenario, the miner chooses the transactions that have higher transaction fees, also based on protocol conditions.

The chosen transactions will be validated and added into the candidate block, which will be mined by the miner.

After preparing the candidate block, the miner starts solving the computationally heavy cryptographic puzzle. The prepared block header is incomplete and waiting for the nonce, which can be identified or discovered by solving the puzzle.

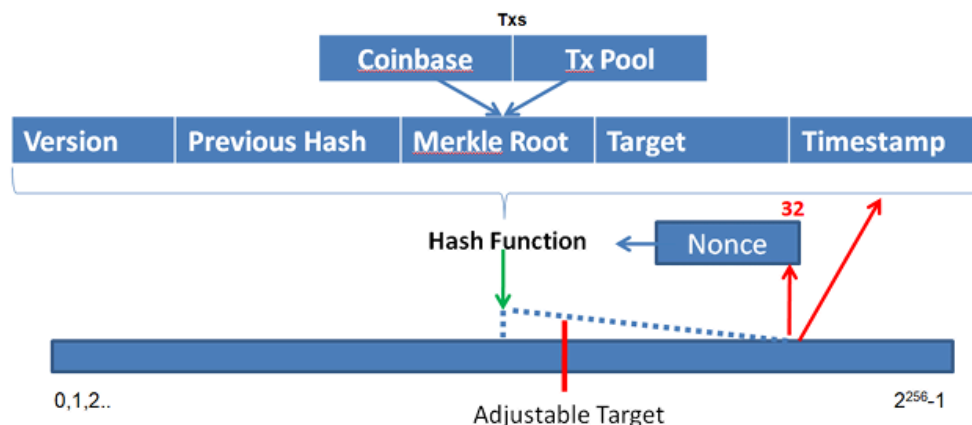
Once the puzzle is solved, the miner fills the nonce into the header. Now, the block is successfully mined by the miner and it will be broadcasted into the network for acceptance.

Every node receiving a block directly from a miner or through any neighbor will validate the transactions, verify the proof of work correctness, and validate the block before acceptance. The receiver verifies the proof of work puzzle solving in negligible time, but solving the puzzle may require exponential time complexity. The miner gets the reward for solving the computationally heavy puzzle, but the verifier will not get any reward since no significant effort is required to verify it. However, the verifier will gain an advantage by proceeding to mine the next successful block.

2.7.1 Cryptographic Puzzle

The PoW protocol forces the miner to solve the computationally heavy cryptographic puzzle to successfully mine the block. Bitcoin miners have to solve a hard problem, but other members can verify the correctness of the solution in just one attempt. The cryptographic

puzzle in PoW is based on cryptographic hash algorithms such as SHA-256 and SHA-512, but SHA-256 is the most widely used.



The SHA-256 algorithm produces an output in the range of 0 to $2^{256} - 1$. If we give any input to the algorithm, it will produce an output value from this range. The puzzle is to find an input that will produce a hash output less than or equal to the target. The target is a value between 0 and $2^{256} - 1$, and it is adjusted according to the protocol implementation. The representation of puzzle solving is shown in Figure 5. The computational difficulty increases when the target value is reduced, and the computational difficulty reduces if the target value is increased. In Bitcoin, the difficulty value is adjusted after every 2016 blocks, approximately every two weeks.

Figure 5: PoW puzzle target adjustment

Figure 6 shows the process of solving the puzzle by repeatedly computing the hash for an input "Hello, World". We consider that the block data is "Hello, World", and accordingly, the puzzle is solved as shown in Figure 6. The values 0, 1, ..., 4250 are the nonce values concatenated with the input data to make the input unique every time. The unique value is required because the cryptographic hash function is deterministic—meaning that no matter how many times we provide the same input, we will get the exact same output. In the example of Figure 6, the target assigned is 2^{240} . The condition of PoW is satisfied when the nonce 4250 is concatenated with "Hello, World". The input "Hello, World" is used for simple representation, but in the actual blockchain, the input includes parameters of the block header as shown in Figure 7.

Every parameter in the block header has a defined bit size. In the case where the block header supports a nonce size of 4 bytes, the maximum value it can support is $2^{32} - 1$. There is a possibility that a miner exhausts all possible 32-bit nonce values (0 to 4,294,967,295) without finding a valid hash. There is no point in repeating the same nonces because they will produce the identical result due to the deterministic nature of the hash function. In such a case, the PoW puzzle cannot be solved to mine the block using the nonce alone.

In such a scenario, the timestamp can be updated, because the timestamp at the beginning of solving the puzzle can vary after trying all possible nonces. It is not required to have a

significant change; any change in the timestamp is sufficient to yield completely new block data and allow the miner to retry all nonces. The other common option is to increment the extra nonce field in the coinbase transaction, depending on the blockchain implementation. The general process of solving the puzzle by modifying the input block data is as follows:

Build the candidate block including the transactions.

Try every nonce value from 0 to $2^{32} - 1$.

If no valid hash is found, repeat the following process to update the input block data:

- Increment the extra nonce in the coinbase transaction and recompute the Merkle root because the transaction data was modified.
-
- Update the timestamp as time advances during the process of trying all possible nonces.
-

2.7.2 Difficulty and Target

The difficulty and target values regulate the rate at which new blocks are added to the blockchain when the Proof of Work (PoW) consensus mechanism is used. The target defines the maximum allowable hash value for a valid block, while the difficulty is inversely proportional to the target; that is, a lower target corresponds to higher difficulty. In Bitcoin, the mining difficulty is adjusted every 2016 blocks (approximately every two weeks, known as a difficulty adjustment period or epoch, assuming 10 minutes per block) based on the difference between the expected and actual time taken to generate those blocks. In Proof of Work-based Ethereum, the mining difficulty was adjusted for every block, aiming to maintain an average block generation time of approximately 13–15 seconds.

Difficulty is used to derive the target value, which proves that the miner expended sufficient computational effort and time to mine the block. A miner must compute a SHA-256 hash output that is less than or equal to the target value.

Bitcoin Difficulty and Target: In Bitcoin, the Difficulty and Target for an epoch are calculated using Equations (1) to (4). There are possibilities that the target and difficulty can suddenly increase or decrease drastically; these unexpected fluctuations are controlled by allowing at most an adjustment by a factor of 4. That is, a new difficulty cannot be more than four times the old difficulty and not less than one-fourth of the old difficulty:

$$(\text{old Difficulty} / 4) \leq \text{Difficulty} \leq (\text{old Difficulty} * 4)$$

$$\text{Difficulty} = \text{Max_Target} / \text{Current Target} \quad \text{--- (1)}$$

Equation (1) shows the computation of difficulty. The Max_Target is fixed in Bitcoin, and the Current Target of the Genesis block is identical to the Max_Target. Hence, the difficulty of the Bitcoin genesis block is 1.

Max_Target =

0010010100111001001000100011001001011110010101100101000000110111

Each block stores an encoded representation of the target called nBits (a 32-bit compact representation) to minimize storage requirements. The encoding and decoding are defined as follows:

$$\text{Target} = \text{Coefficient} * 256^{(\text{Exponent} - 3)}$$

or equivalently,

$$\text{Target} = \text{Coefficient} * 2^{(8 * (\text{Exponent} - 3))}$$

The block header does not need to store the entire 256-bit value; it only needs to store:

Coefficient = significant digits (24 bits / 3 bytes)

Exponent = number of bytes (8 bits / 1 byte)

Hence,

$$\text{nBits} = [\text{Exponent (1 byte)} \mid \text{Coefficient (3 bytes)}]$$

The base 256 (or 2^8) and the subtraction value 3 are not stored in nBits because they are constants fixed by the Bitcoin protocol. During encoding and decoding, nodes compute $\text{Coefficient} * 256^{(\text{Exponent} - 3)}$ to reconstruct the full 256-bit target. For example, the Max_Target is encoded as 0x1d00ffff in 32 bits instead of 256 bits. The decoding mechanism of the 32-bit compact target to the actual 256-bit target is given below:

The computation of the Current Target (the target for the upcoming 2016 blocks) will be calculated using Equations (2) to (4). First, the deviation between the expected time to mine 2016 blocks and the actual time taken will be computed:

$$\text{Deviation} = \text{Expected Time} / \text{Actual Time} \text{ --- (2)}$$

$$\text{New Difficulty} = \text{Deviation} * \text{old Difficulty} \text{ --- (3)}$$

$$\text{Current Target} = \text{Max_Target} / \text{Difficulty} \text{ --- (4)}$$

Using the New Difficulty and the Max_Target, the Current Target will be computed as shown in Equation (4). The old Difficulty is the difficulty of the previous 2016 blocks. As defined, approximately 10 minutes are targeted to mine a Bitcoin block, but the actual time varies based on global hash power.

3.2 Asymmetric Key Cryptography

An asymmetric-key cryptosystem, also called public key cryptography, includes encryption and decryption operations. However, unlike a symmetric-key cryptosystem, it uses two different but mathematically related keys: a public key and a private key. Figure 12 shows the flow of a message in an asymmetric or public key cryptography process with two keys: one to encrypt plaintext to produce ciphertext, and the other to decrypt ciphertext to produce plaintext.

For example, receiver Bob generates the key pair (K_{Pub} , K_{Pri}) and shares the public key K_{Pub} with sender Alice. Alice uses the public key (K_{Pub}) of Bob for encryption of the plaintext message. The encrypted message in the form of ciphertext is sent to Bob. Now, Bob uses his private key (K_{Pri}) for the decryption process, which decrypts the ciphertext and produces the plaintext.

Figure 12: Asymmetric Key Cryptography

The public key and the private key are mathematically related and complement each other. The two keys perform inverse cryptographic operations. A message encrypted with the public key can be decrypted only with the corresponding private key, and vice versa in applications such as digital signatures. Although the keys are mathematically related, it is computationally infeasible to derive the private key from the public key, which forms the basis for the security of asymmetric cryptography.

As the name suggests, the public key is intended to be publicly available and can be freely distributed to anyone. In contrast, the private key must remain secret and should be known only to its owner. The security of an asymmetric-key cryptosystem depends on protecting the private key while allowing unrestricted access to the corresponding public key. Several asymmetric key algorithms have been developed, including RSA, Elliptic Curve Cryptography (ECC), and the Diffie–Hellman key exchange algorithm. RSA and ECC are commonly used for encryption and digital signatures, whereas Diffie–Hellman is primarily used for secure key exchange between communicating parties.

Asymmetric cryptography is complex compared to symmetric cryptography. In the case of symmetric key cryptography, it is mostly permutation, substitution, and mixing, but the asymmetric cryptosystem involves complex mathematical computations.

The properties of public key or asymmetric key algorithms are as follows:

Confidentiality: Public key cryptography ensures confidentiality provided that the private key is held only by the intended recipient while the public key is known to anyone, including the sender. No one can decipher the message without knowing the private key.

Authentication: The asymmetric cryptosystem can achieve authentication by performing encryption with the private key, which is known exclusively to the sender. Anyone possessing the corresponding public key can verify authenticity, since only

the sender possesses that respective private key. Digital signatures combined with asymmetric cryptosystems are used today to achieve authenticity.

Message Integrity: Integrity is preserved when the private key is used to sign or encrypt the digest of the message. In the case of using purely a public key for encryption, integrity cannot be guaranteed alone since the public key is accessible to everyone, allowing an adversary to apply malleable encryption to modify the message without detection. Malleable encryption is an encryption property that lets an unauthorized party alter an encrypted message so that, when decrypted, the message changes in a predictable way. The attacker does not need to know the original text or the secret key to make this change. In an unauthenticated asymmetric cryptosystem, an attacker knowing the public key can alter the message and it will be accepted by the receiver.

Key Usage: The public key cryptosystem includes both a private key and a public key. These keys are used for encryption and decryption according to the application requirement. The public key is used for encryption and the private key for decryption when confidentiality is required, because anyone can encrypt the message using the public key, but decryption can only be performed by the private key holder (the intended recipient). Conversely, the private key is used for encryption and the public key for decryption in digital signatures, where we require only the owner to sign the message while allowing any party to verify it.

Key Management: In asymmetric-key cryptography, each user generates a pair of keys:

Public key: Shared openly with everyone through the public channel or network without any encryption.

Private key: Kept secret by the owner.

If there are n users in the network, each user possesses one public key and one private key. Therefore, the total number of keys is given by:

$$K = 2n \text{ --- (6)}$$

For example, if there are 100 users:

$$K = 2 * 100 = 200 \text{ keys. --- (7)}$$

However, since public keys are intended to be publicly available and do not require confidentiality, the actual secret key management burden is only:

$$K_{\text{private}} = n \text{ --- (8)}$$

For 100 users:

$$K_{\text{private}} = 100 \text{ private keys. --- (9)}$$

In symmetric-key cryptography, the number of secret keys grows quadratically as $n(n - 1) / 2$. Asymmetric key cryptography requires each user to manage only a single private key, significantly simplifying key management. However, asymmetric key sizes are considerably larger compared to symmetric key sizes to ensure equivalent security levels.

3.2.1 The Asymmetric Bitwise XOR Cryptosystem

In theoretical and educational abstractions of asymmetric primitives, bitwise operations are sometimes evaluated to illustrate the dual-key concept. In an asymmetric XOR cryptosystem, encryption and decryption rely on the algebraic involution of the exclusive-OR (XOR) operator over binary blocks.

In this scheme, the user defines a shared base key vector or modulus parameter, splitting key components into an asymmetric pair: a public key bitstring K_{Pub} and a mathematically related private key bitstring K_{Pri} .

Mathematical Representation and Flow

Let the plaintext message represented in binary form be M , the ciphertext be C , the public key bitstring be K_{Pub} , and the private key bitstring be K_{Pri} .

Key Generation: A secret master entropy string K_{Master} is selected. The key pair is generated such that the bitwise combination satisfies an inverse relationship:

$$K_{Pub} \text{ XOR } K_{Pri} = \Delta$$

where Δ is a known system-wide transformation constant or mask.

Encryption: The sender obtains Bob's public key K_{Pub} and computes the ciphertext C by executing the XOR operation across the plaintext bitstream:

$$C = M \text{ XOR } K_{Pub}$$

Decryption: The receiver (Bob) decrypts the incoming ciphertext C using his private key K_{Pri} alongside the inverse mask Δ :

$$M = C \text{ XOR } K_{Pri} \text{ XOR } \Delta$$

Because the XOR operator is associative and commutative ($A \text{ XOR } B \text{ XOR } A = B$), applying the private key cancels out the public key transformation:

$$M = (M \text{ XOR } K_{Pub}) \text{ XOR } K_{Pri} \text{ XOR } \Delta = M \text{ XOR } (K_{Pub} \text{ XOR } K_{Pri}) \text{ XOR } \Delta = M \text{ XOR } \Delta \text{ XOR } \Delta = M$$

Cryptanalysis and Inherent Vulnerabilities

While the bitwise XOR method is computationally lightweight and simple to implement, it is fundamentally flawed and entirely unsafe for practical cryptographic operations. The security vulnerabilities of the XOR cryptosystem are detailed below:

Linearity and Lack of One-Way Trapdoors: Unlike true asymmetric algorithms, the XOR function possesses no trapdoor one-way permutation properties. Because the relationship $K_{Pub} \text{ XOR } K_{Pri} = \Delta$ is entirely linear, an adversary who intercepts the public key K_{Pub} can trivially compute the private key:

$$K_{Pri} = K_{Pub} \text{ XOR } \Delta$$

This completely violates the foundational rule of public key cryptography: that deriving the private key from the public key must be computationally infeasible.

Known-Plaintext Vulnerability: If an attacker intercepts both the ciphertext C and a portion of the original message M, the key can be instantly recovered with zero computational overhead:

$$K_{\text{Pub}} = C \text{ XOR } M$$

Lack of Diffusion and Confusion: XOR provides purely linear substitution without diffusion. A change of a single bit in the plaintext produces a change in exactly one corresponding bit in the ciphertext, making it completely vulnerable to frequency analysis, bit-flipping attacks, and differential cryptanalysis.

Justification for Modern Asymmetric Cryptosystems (RSA)

Due to these catastrophic vulnerabilities, naive linear algebraic schemes like the XOR method are never deployed in production blockchains or secure distributed ledgers. Instead, robust cryptosystems such as the RSA Algorithm are mandatory.

RSA achieves true one-way asymmetric security by grounding its mathematical trapdoor in the Integer Factorization Problem—a non-linear, mathematically hard problem where multiplying two large prime numbers (p and q) to generate a modulus $n = p * q$ is computationally trivial, but deriving the private prime factors from the publicly exposed modulus n is computationally infeasible with classical computing architectures.